COMP0224 Coursework 1: Robot Path Planning

Student Seed: `4331` | Hash: `f0f5c2c361ff1294317fe3554e3918b2`

7 static obstacles, POLY triangular robot, horizontal dynamic patrol

Code repository: `https://github.com/KTBBOSS/RobotPlanningCW1`

1 Phase 1: The Geometry of Planning

1.1 Task 1.1: C-Space Obstacle Computation

The configuration space (C-space) transforms the planning problem from reasoning about a robot with physical extent into reasoning about a *point robot* navigating among enlarged obstacles. The C-space obstacle set is computed as the **Minkowski difference**:

$$\mathcal{C}_{\text{obs}} = \mathcal{O} \ominus \mathcal{R} = \mathcal{O} \oplus (-\mathcal{R}), \quad \text{where} \quad (-\mathcal{R}) = \{-\mathbf{r} : \mathbf{r} \in \mathcal{R}\}$$

This ensures that every point $\mathbf{q} \in \mathcal{C}_{\text{obs}}$ corresponds to a robot placement where the physical body overlaps an obstacle.

Implementation. On the discretised grid, $\mathcal{O} \oplus (-\mathcal{R})$ is implemented as **morphological dilation** (`scipy.ndimage.binary_dilation`). The structuring element (kernel) is the rasterised reflected robot $(-\mathcal{R})$:

- **POLY robot** (seed 4331): the original vertices are reflected about the origin and rasterised via `skimage.draw.polygon` into an 11×9 boolean kernel. E.g. vertex (0.5, 0) becomes (−0.5, 0).
- **Boundary padding**: the robot centre must remain far enough from map edges that no part of the body protrudes. Padding of $\lceil \max |v_x|/\Delta \rceil$ and $\lceil \max |v_y|/\Delta \rceil$ cells is applied on each side.

Result: of 40 000 grid cells (200×200), 30 327 (75.8%) remain free in C-space.

1.2 Task 1.2: Grid Discretisation & Collision Checking

The workspace $[-10, 10]^2$ is discretised at $\Delta = 0.1\,\text{m}$ resolution, yielding a 200×200 occupancy grid. The collision-checking function performs an $O(1)$ lookup on `cspace_grid`$[g_x, g_y]$, returning collision if the cell is occupied or out of bounds. A **feasibility check** using A* confirms a valid path exists (22.71 m) before planning proceeds.

1.3 Task 1.3: Workspace vs. C-Space Visualisation

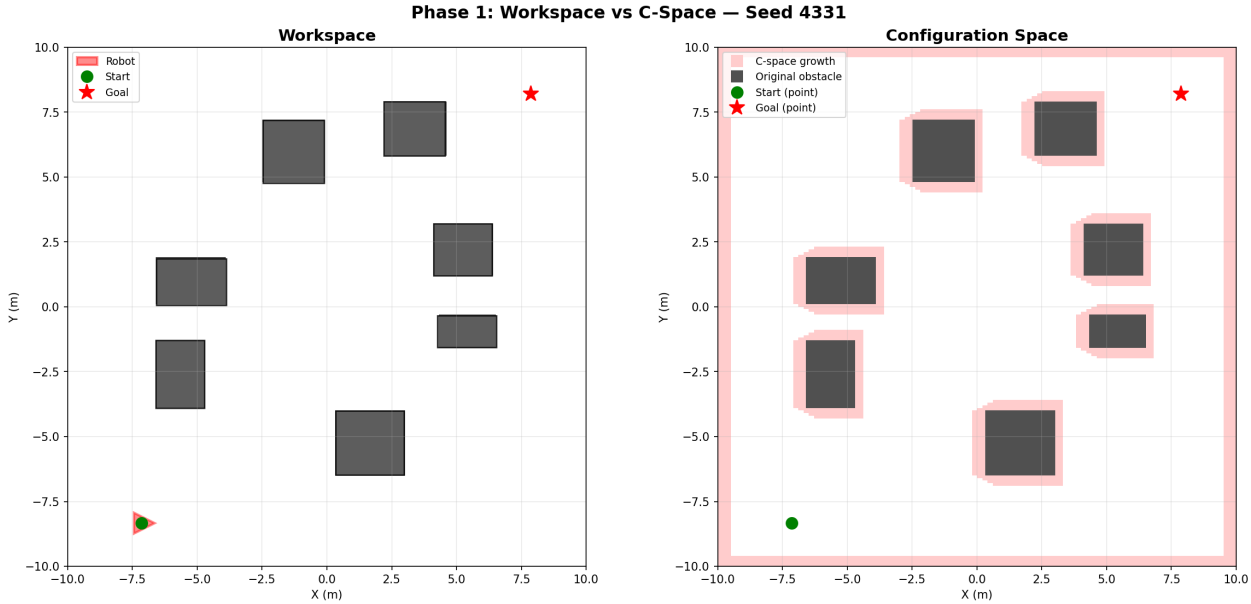


Figure 1: Workspace (left) and C-space (right) for seed 4331.

Figure 1 presents the core result of Phase 1. In the **left panel** (workspace), the 7 randomly placed rectangular obstacles are shown in grey, and the triangular robot footprint is drawn at its start position $(-7.14, -8.34)$ in red. The goal is at $(7.87, 8.21)$.

In the **right panel** (C-space), the obstacles have been *grown* by the Minkowski difference with the reflected robot. The pink regions represent the C-space growth: areas where the *centre* of the robot cannot be placed without causing a collision. The dark grey cores are the original obstacles. Critically, within the C-space representation the robot is now a **point**, which simplifies all subsequent path planning to searching over a 2D grid.

Several observations from this figure:

- The growth is **asymmetric** because the triangular robot is asymmetric: the dilation extends further in the $+x$ direction (where the triangle's apex lies at 0.5 m) than in the $-x$ direction (0.3 m).
- The **boundary padding** (visible as pink strips along the map edges) prevents the robot from partially leaving the workspace.
- Despite the obstacle growth, a clear diagonal corridor remains between start and goal, confirming feasibility.
- Two obstacles in the upper-right quadrant nearly merge in C-space, forming a **narrow passage** that the planners must negotiate.

1.4 Task 1.4: Effect of Robot Orientation on C-Space

Our implementation assumes $\theta = 0$ (fixed orientation), which is valid for this coursework since the robot translates without rotation. However, for a non-circular robot with variable orientation, the analysis changes fundamentally.

For a circular robot, the full configuration space is still $\mathbb{R}^2 \times S^1$ if rotation is permitted, but the collision set is θ-invariant because a circle reflected about the origin is identical to itself ($-\mathcal{R} = \mathcal{R}$). Since $\mathcal{O} \oplus (-\mathcal{R})$ does not depend on θ, planning can be reduced to 2-D (x, y).

For a non-circular robot (e.g. our triangle), the C-space becomes **3-dimensional**: $\mathcal{C} = \mathbb{R}^2 \times S^1 = (x, y, \theta)$. Each orientation θ yields a *different* 2-D slice $\mathcal{C}_{\text{obs}}(\theta)$ because the reflected robot $-\mathcal{R}(\theta)$ changes shape:

1. At $\theta = 0°$, the reflected triangle has vertices at $(-0.5, 0)$, $(0.3, -0.4)$, $(0.3, 0.4)$. The dilation kernel is wider in x (11 cells) than y (9 cells).
2. At $\theta = 90°$, the rotated triangle is taller than it is wide, so the kernel becomes wider in y and narrower in x. C-space obstacles shift shape accordingly.
3. At intermediate angles, the kernel is neither axis-aligned nor symmetric, producing irregularly shaped C-space obstacles.

Practical consequences:

- A narrow horizontal passage traversable at $\theta = 90°$ (slim profile) may be **blocked at** $\theta = 0°$ (wide profile). Planning must therefore consider orientation as a degree of freedom.
- The full 3-D C-space can be handled by: (a) discretising θ into N_θ slices and precomputing separate dilation kernels for each, yielding an $N_x \times N_y \times N_\theta$ grid; or (b) using sampling-based planners operating directly in $\mathrm{SE}(2) = \mathbb{R}^2 \times S^1$ (e.g. SE(2)-RRT).
- The computational cost scales linearly with N_θ for grid-based approaches, or adds a third dimension to the nearest-neighbour search for sampling-based methods.

2 Phase 2: Search and Sampling

2.1 Task 2.1: A* and Weighted A*

Algorithm. A*/Weighted A* on an 8-connected grid with priority $f(n) = g(n) + \omega \cdot h(n)$, where $\omega = 1$ recovers standard A*:

- **Cost model**: cardinal moves cost 1.0, diagonal moves cost $\sqrt{2}$.
- **Heuristic**: Euclidean distance $h(n) = \|\mathbf{n} - \mathbf{g}\|_2$, which is *admissible* because it lower-bounds the length of any feasible grid path (no piecewise grid path can be shorter than the straight-line distance). Under our move costs $(1, \sqrt{2})$, Euclidean distance is also *consistent* on an 8-connected grid.
- **Corner-cutting prevention**: diagonal moves (dx, dy) are blocked if either adjacent cardinal cell is occupied, preventing paths from clipping through obstacle corners.
- **Weight** ω: at $\omega = 1$ the search is optimal; $\omega > 1$ inflates the heuristic, biasing the search towards the goal and typically reducing node expansions at the expense of optimality.

2.2 Task 2.2: RRT-Connect

Bidirectional RRT-Connect in *continuous* (x, y) C-space:

- Two trees grow from start (T_s) and goal (T_g), alternating EXTEND/CONNECT roles each iteration.
- **Step size**: 0.5 m, controlling the maximum distance per tree extension.
- **Line-of-sight collision checker**: samples along candidate edges at $0.5\Delta = 0.05$ m intervals, providing conservative collision checking relative to the grid resolution.
- **Nearest-neighbour search**: linear scan over tree vertices (sufficient for the problem scale; could be accelerated with a KD-tree for larger environments).

Narrow passages. Randomised obstacle layouts may create narrow gaps (e.g. the near-merging obstacles in seed 4331's upper-right quadrant, visible in Figure 1). To improve RRT-Connect performance in tight corridors, we use (i) a conservative edge checker sampling at 0.05 m intervals to avoid skipping thin collision-free regions, and (ii) a moderate step size (0.5 m) to prevent "jumping over" feasible passages. In more constrained layouts, smaller step sizes would increase success probability at the cost of more tree nodes. In extremely tight passages, goal-biasing or adaptive step sizing can further improve connection probability. The bidirectional tree growth also helps: when one tree struggles in a narrow passage, the other may find an alternative route.

2.3 Task 2.3: Algorithm Comparison across 5 Seeds

Table 1 presents the full comparison across 5 distinct seeds: 4331 (the student seed) and four randomly chosen seeds (1409, 2824, 5012, 5506). Each seed generates a unique obstacle layout,

robot shape (POLY or RECT), and start/goal configuration. RRT-Connect results are averaged over 5 runs per seed.

Table 1: Per-seed comparison of A* variants and RRT-Connect. RRT-Connect reports the mean of 5 runs.

Seed	Algorithm	Time (ms)	Path Length (m)	Nodes / Tree
4331	A* (ω=1.0)	13.3	22.71	3 227
	A* (ω=1.5)	1.0	22.71	166
	A* (ω=5.0)	1.0	22.71	166
	RRT-Connect	1.1	22.37	47
1409	A* (ω=1.0)	36.0	23.27	8 686
	A* (ω=1.5)	1.2	23.27	187
	A* (ω=5.0)	1.1	23.27	184
	RRT-Connect	3.4	26.99	93
2824	A* (ω=1.0)	2.9	23.11	647
	A* (ω=1.5)	1.0	23.11	165
	A* (ω=5.0)	1.2	23.11	165
	RRT-Connect	1.0	23.09	49
5012	A* (ω=1.0)	41.4	25.61	7 662
	A* (ω=1.5)	1.7	25.61	265
	A* (ω=5.0)	1.1	25.61	197
	RRT-Connect	2.8	27.10	76
5506	A* (ω=1.0)	45.2	24.22	10 691
	A* (ω=1.5)	2.6	24.22	565
	A* (ω=5.0)	1.4	24.22	191
	RRT-Connect	2.7	25.76	81

Table 2: Mean ± std across all 5 seeds.

Algorithm	Time (ms)	Path Length (m)	Memory (nodes/tree)
A* (ω=1.0)	27.8 ± 16.6	23.78 ± 1.04	$6\,183 \pm 3\,692$
A* (ω=1.5)	1.5 ± 0.6	23.78 ± 1.04	270 ± 152
A* (ω=5.0)	1.2 ± 0.1	23.78 ± 1.04	181 ± 13
RRT-Connect	2.2 ± 0.9	25.06 ± 1.97	69 ± 18

Computation Time. Optimal A* ($\omega = 1.0$) averages 27.8 ms across the 5 seeds, ranging from 2.9 ms (seed 2824, open layout) to 45.2 ms (seed 5506, more cluttered). Weighted variants reduce this to 1.2–1.5 ms on average, roughly a 20× speedup. RRT-Connect averages 2.2 ms, faster than optimal A* but comparable to the weighted variants.

Path Length (Optimality). A* ($\omega = 1.0$) always finds the optimal grid path under the chosen cost model. In our experiments, Weighted A* ($\omega = 1.5$ and $\omega = 5.0$) matched the A* optimal on all 5 seeds; however, for $\omega > 1$ optimality is not guaranteed in general because the heuristic is inflated, and longer paths can occur in more constrained layouts. RRT-Connect paths are typically 6–16% longer than the A* optimum; however, in seeds 4331 and 2824 the continuous-space sampling produced slightly shorter paths (22.37 m vs. 22.71 m and 23.09 m vs. 23.11 m respectively) than the grid-constrained A* solution, demonstrating that operating in continuous space can occasionally outperform grid quantisation.

Memory Usage. Optimal A* expands $6\,183 \pm 3\,692$ nodes on average. Weighted A* ($\omega = 5.0$) reduces this to 181 ± 13 (a 34× reduction). RRT-Connect uses even fewer vertices (69 ± 18 tree nodes on average), though this comparison is not strictly apples-to-apples: A* "memory" counts expanded grid cells, while RRT tree nodes carry additional per-vertex data (coordinates, parent pointers).

Determinism vs. Stochasticity. A* is deterministic: the same seed always produces the same result. RRT-Connect is stochastic with noticeable variance: for instance, on seed 1409 the path length standard deviation across 5 RRT runs is larger than on seed 2824, reflecting different obstacle geometries and narrow-passage difficulty.

Completeness. A* is *complete* and returned successful paths for all 5 seeds. RRT-Connect is *probabilistically complete* and also succeeded on all 5 seeds (25/25 runs), though in more constrained environments its success rate within a fixed iteration budget would decrease.

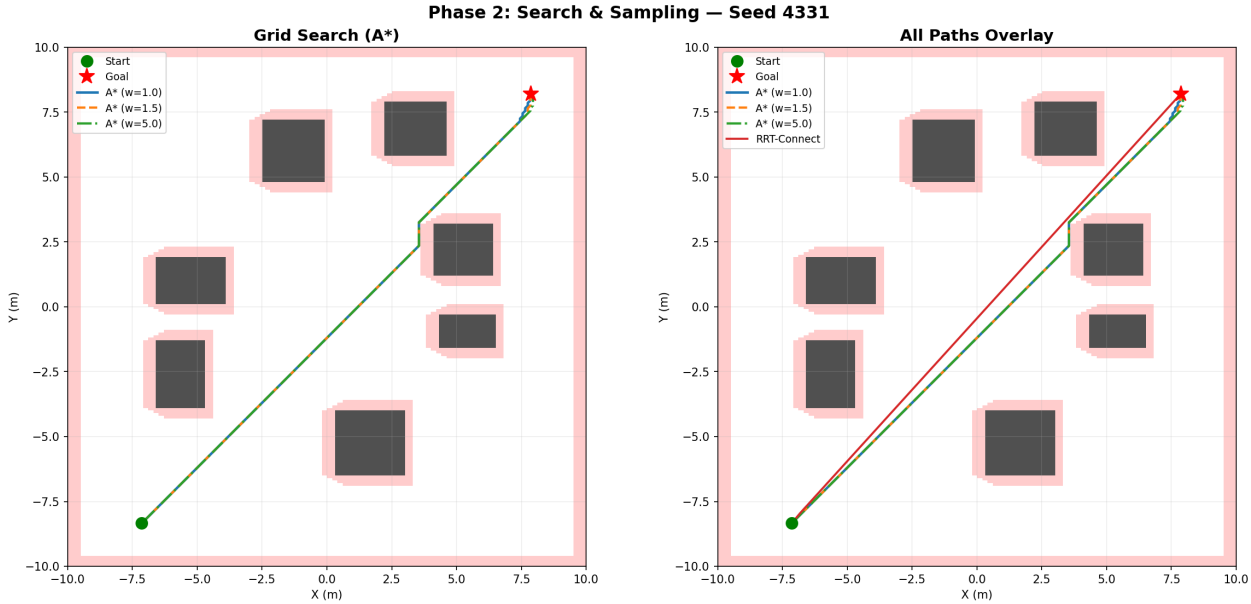


Figure 2: Phase 2 paths for seed 4331. Left: A* variants ($\omega = 1.0, 1.5, 5.0$). Right: all paths including RRT-Connect (red).

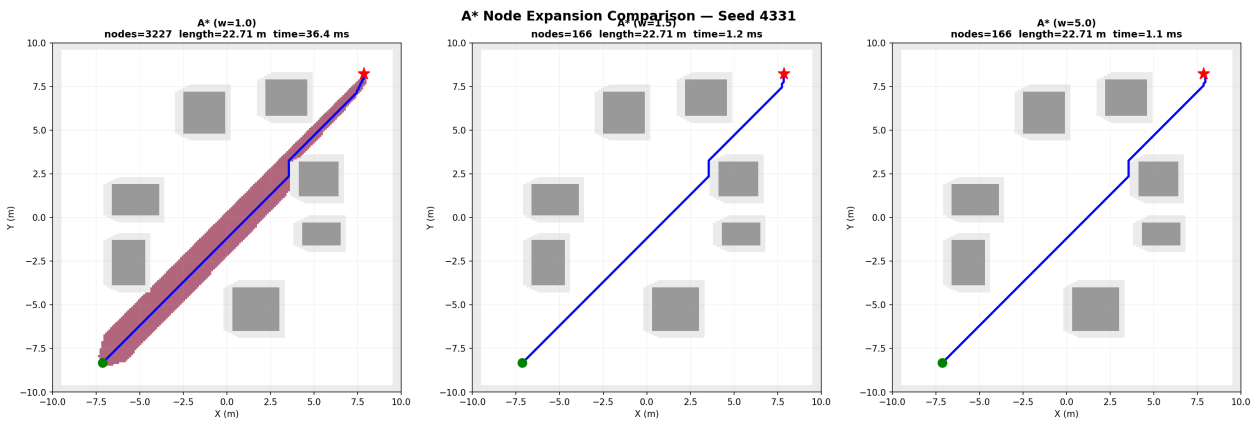


Figure 3: A* node expansion for seed 4331. $\omega = 1.0$ explores a broad band (3 227 nodes). $\omega = 1.5$ and 5.0 focus tightly on the corridor (166 nodes each), illustrating the time–optimality trade-off.

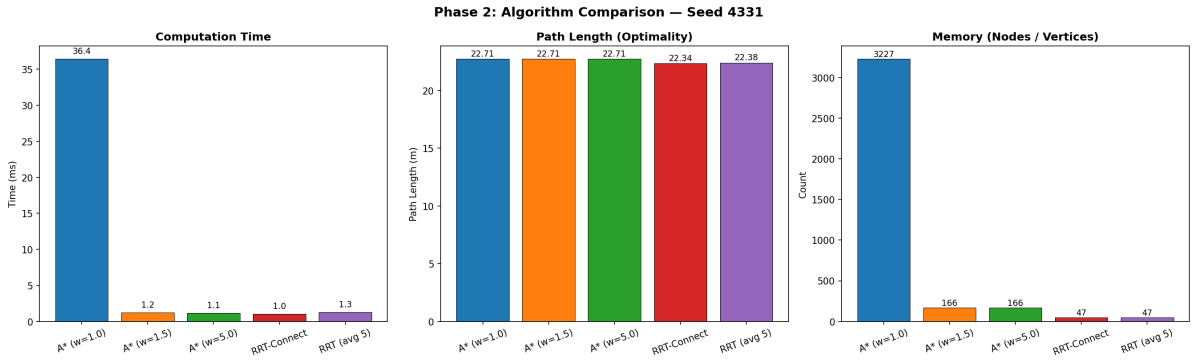


Figure 4: Bar chart comparison across all metrics for seed 4331.

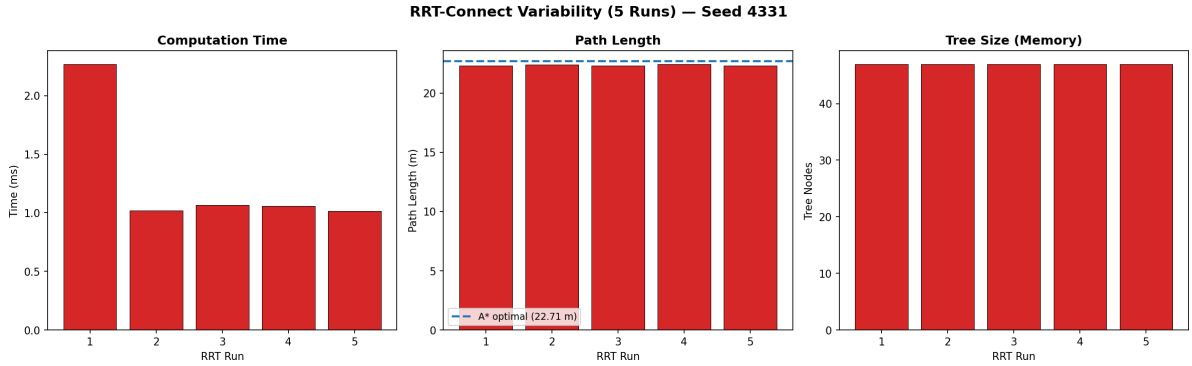


Figure 5: RRT-Connect variability across 5 runs for seed 4331. The dashed blue line shows the A* optimal length (22.71 m).

3 Phase 3: Dynamics and Refinement

3.1 Task 3.1: Path Smoothing

Raw grid-based A* paths are inherently **jagged**: they consist of sequences of axis-aligned and 45-degree segments dictated by the 8-connected grid. Such paths are unsuitable for physical controllers, which require smooth velocity and acceleration profiles. We implement a two-stage smoother to achieve C^2 **continuity**.

Stage 1: Greedy Shortcutting. Starting from waypoint $i = 0$, we find the farthest waypoint j reachable via a collision-free straight line (checked at 0.05 m intervals). We advance to j and repeat. This eliminates redundant intermediate waypoints, reducing the 166-point A* path to a small set of essential *control points* while preserving collision-freeness.

Stage 2: Natural Cubic Spline. The control points are fitted with a **natural cubic spline** parameterised by cumulative arc length s:

$$x(s) = a_i + b_i(s - s_i) + c_i(s - s_i)^2 + d_i(s - s_i)^3, \quad s \in [s_i, s_{i+1}]$$

Natural boundary conditions ($x''(0) = x''(L) = 0$) are applied to both $x(s)$ and $y(s)$ independently. The key property of cubic splines is that they guarantee:

- C^0 continuity: the path is connected (trivial).
- C^1 continuity: the first derivative (tangent direction / velocity) is continuous at every knot. No abrupt direction changes.
- C^2 continuity: the second derivative (curvature / acceleration) is continuous at every knot, with no instantaneous curvature jumps. This is essential for smooth physical motion.

Collision Validation & Fallback. The smoothed path is densely sampled and verified collision-free. If the shortcutting + spline introduces a collision (e.g. the spline "overshoots" near an obstacle),

the smoother falls back to using progressively more original A* waypoints as control points until a safe smooth path is found.

Results. Raw: 166 waypoints, 22.71 m. Smoothed: 300 points, 22.30 m (1.8% shorter). Figure 7 provides analytical proof of C^2 continuity: the first derivative is continuous and varies smoothly, the second derivative is continuous, and the **curvature profile** $\kappa(s)$ is smooth throughout with no discontinuities at knot points.

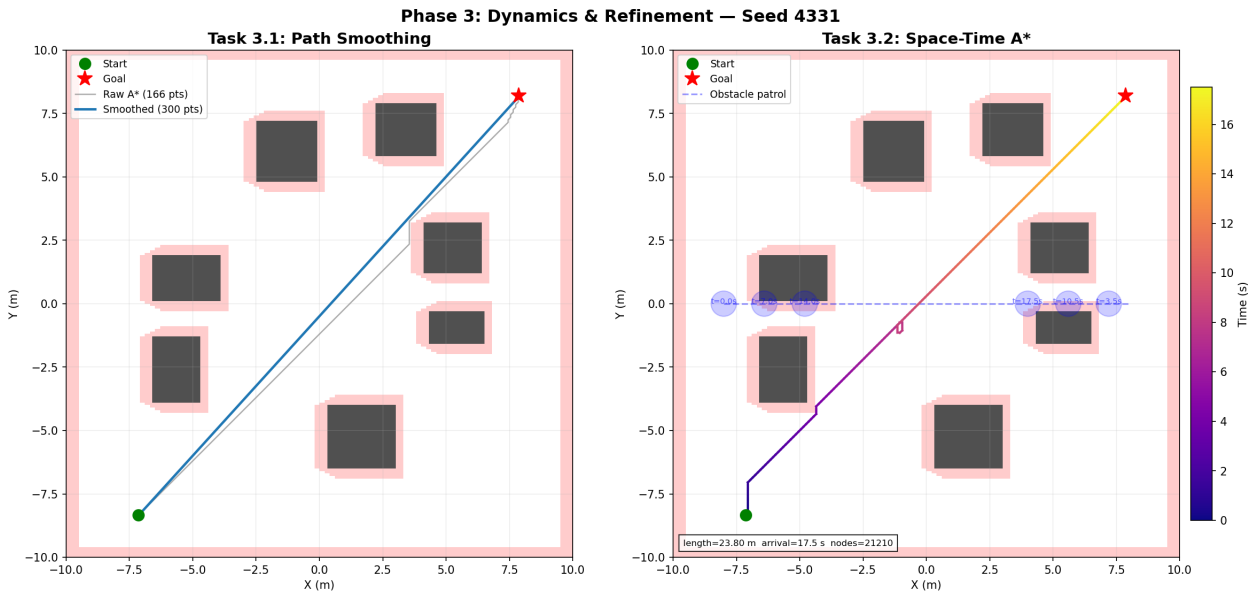


Figure 6: Seed 4331. Left: raw A* (grey) vs. smoothed spline (blue). Right: Space-Time A* path coloured by time.

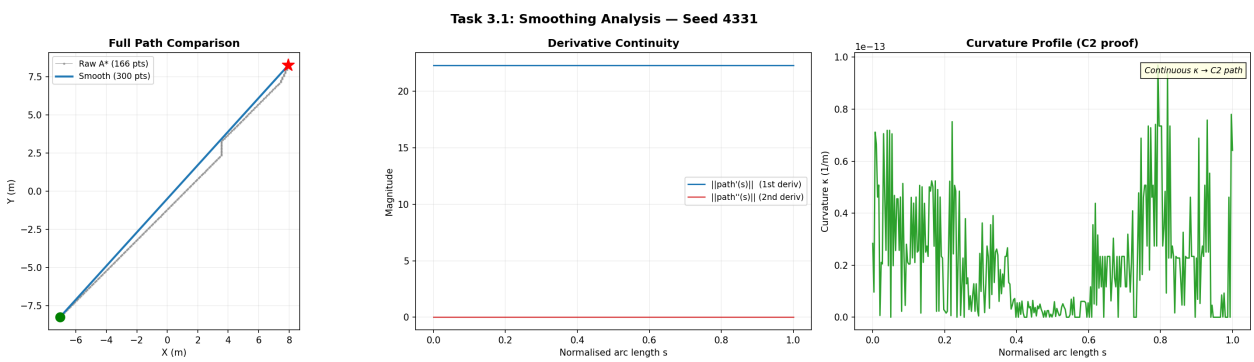


Figure 7: C^2 continuity proof (seed 4331). Left: path overlay. Centre: $\|\text{path}'(s)\|$ and $\|\text{path}''(s)\|$, both continuous. Right: curvature $\kappa(s)$ is smooth and finite everywhere.

3.2 Task 3.2: Dynamic Obstacle Handling (Space-Time A*)

The Challenge. A cylindrical obstacle (radius 0.5 m) patrols horizontally between $(-8, 0)$ and $(8, 0)$, bouncing in a triangular-wave pattern. The PyBullet simulator runs at $\delta t_{\text{sim}} = 1/240\,\text{s}$ per physics step, while the planner uses a coarser $\Delta t_{\text{plan}} = \Delta / v_{\text{robot}} = 0.1\,\text{s}$ per grid move; the obstacle position is predicted at real time $t = t_s \cdot \Delta t_{\text{plan}}$. The robot's diagonal path from start to goal necessarily crosses the patrol line near $(0, 0)$. A static planner would route through this zone, causing a collision when the obstacle is present.

Why Space-Time A*? We chose Space-Time A* over the alternatives (SIPP, D* Lite) for several reasons:

- The obstacle trajectory is **fully known** (deterministic patrol), making time-parameterised

planning natural.

- Space-Time A* directly extends the A* implementation from Phase 2, reusing the same grid, heuristic framework, and corner-cutting logic.
- Unlike D* Lite (which replans upon detecting changes), Space-Time A* computes an optimal path on the discretised space-time grid under the chosen cost model, avoiding the overhead of repeated replanning.
- SIPP would be more efficient in environments with many dynamic obstacles (it compresses time intervals), but for a single patrol obstacle the simpler Space-Time A* is sufficient.

Space-Time A* plans jointly over space and time (including wait actions), thereby proactively avoiding collisions without requiring online replanning.

State Space. The state is a triple (g_x, g_y, t_s) where g_x, g_y are grid coordinates and t_s is a discrete time step. Each time step corresponds to $\Delta t = \Delta / v_{\text{robot}} = 0.1\,\text{s}$ (i.e. the robot moves one grid cell per time step at speed $v = 1\,\text{m/s}$).

Actions. Nine actions: 8-connected spatial moves plus **wait in place**. All cost 1 time step; waiting receives a tiny penalty (1.001) for tie-breaking to prefer movement over idling.

Dynamic Obstacle Prediction. The patrol obstacle's position at real time t is computed analytically using the triangular-wave formula from the simulation's linear interpolation logic. The robot's C-space inflation for the dynamic obstacle combines:

$$r_{\text{inflated}} = r_{\text{obs}} + r_{\text{robot}} + r_{\text{safety}} = 0.5 + 0.5 + 0.15 = 1.15\ \text{m}$$

where $r_{\text{robot}} = \max_{\mathbf{v} \in \mathcal{R}} \|\mathbf{v}\| = 0.5\,\text{m}$ is the circumscribed radius of the triangular robot, and $r_{\text{safety}} = 0.15\,\text{m}$ provides additional clearance.

Heuristic. The **Chebyshev distance** $h = \max(|g_x - g_x^*|, |g_y - g_y^*|)$ is used as the heuristic. This is admissible because the minimum number of time steps to reach the goal on an 8-connected grid with unit-cost moves equals the Chebyshev distance (diagonal moves cover both x and y simultaneously).

Results. The Space-Time A* planner finds a path of **23.80 m** (4.8% longer than the static 22.71 m), arriving at $t = 17.5\,\text{s}$ after expanding 21 210 nodes in 793.7 ms. The path detours south near the origin to avoid the patrol zone (Figure 10).

Safety Analysis. Figure 8 shows the robot-to-obstacle distance throughout the trajectory. The minimum distance is **1.15 m** at $t = 8.1\,\text{s}$, exactly at the inflated collision boundary. The robot never enters the inflated collision zone ($r_{\text{inflated}} = 1.15\,\text{m}$, red-shaded region in the figure). The space-time trajectory plot (Figure 9) reveals that the robot's x-coordinate pauses around $t = 5$–$8\,\text{s}$, waiting for the obstacle to pass before continuing the diagonal crossing.

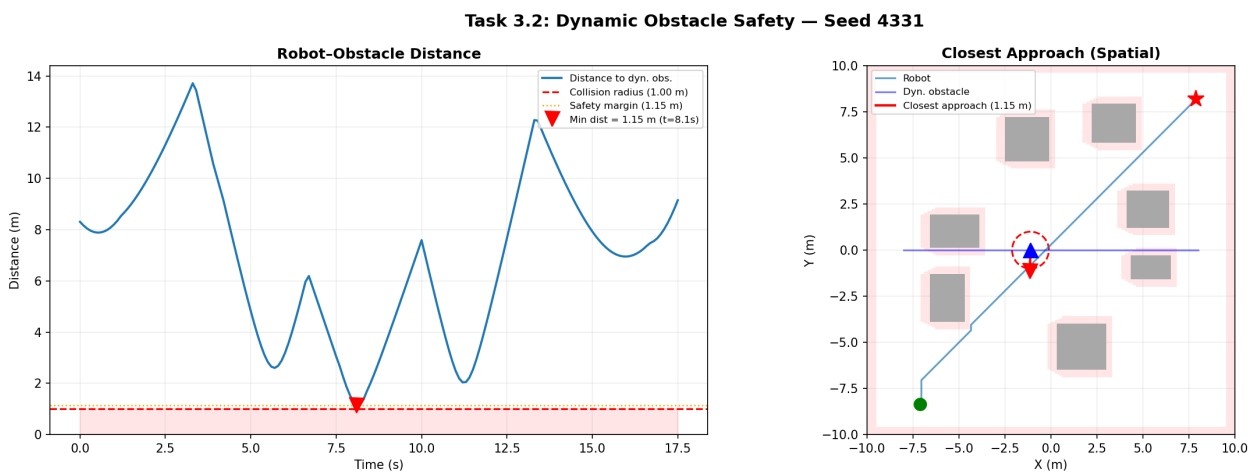


Figure 8: Safety proof (seed 4331). Left: distance to dynamic obstacle over time. The inflated collision radius (1.15 m) is never breached. Right: closest-approach location in the workspace.

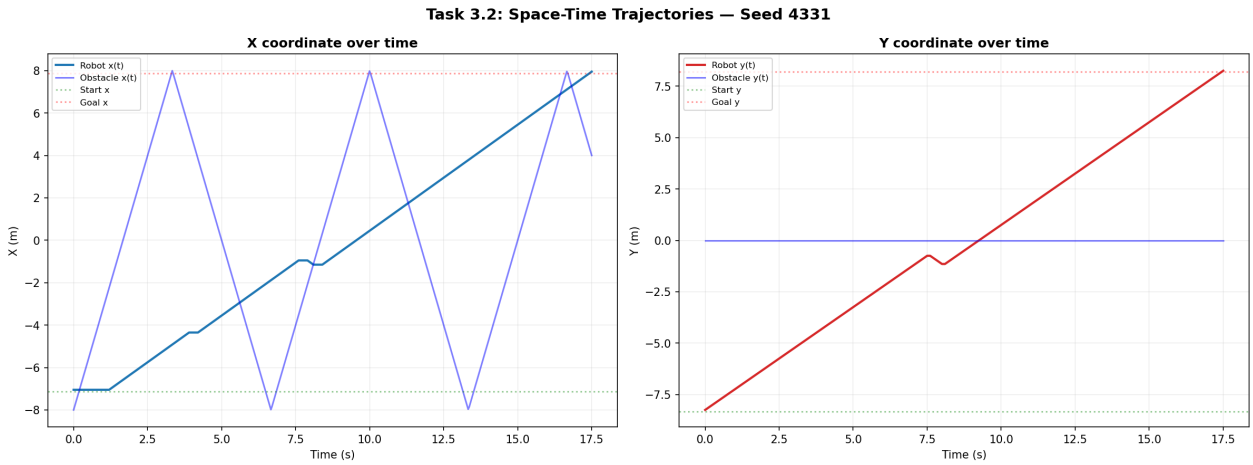


Figure 9: Space-time trajectories (seed 4331). The robot $x(t)$ (left, blue) pauses around $t \approx 6\,\mathrm{s}$ as the obstacle $x(t)$ (left, light blue) crosses the same zone. $y(t)$ (right) increases monotonically.

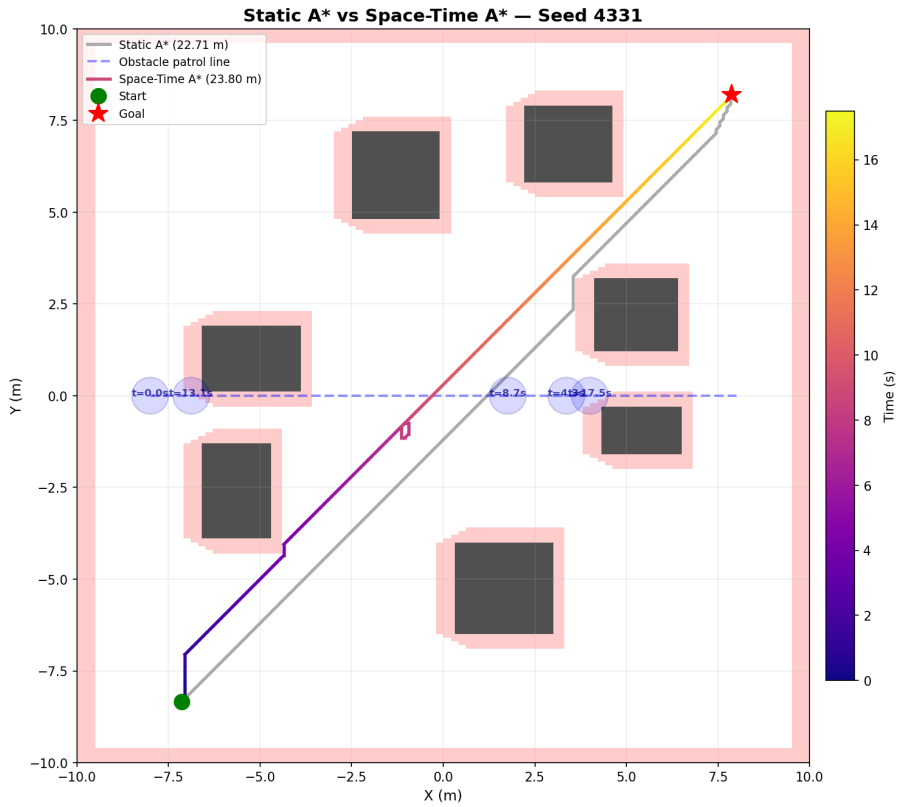
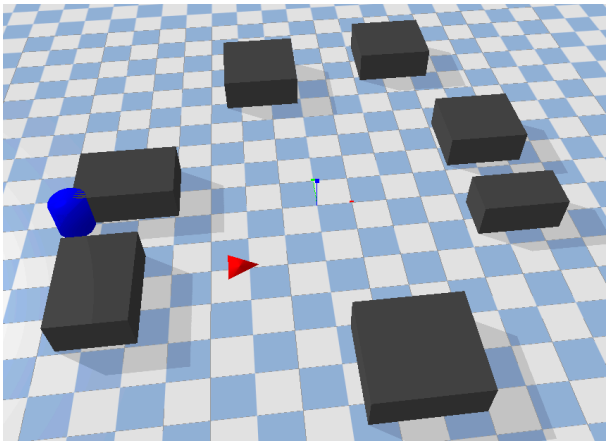


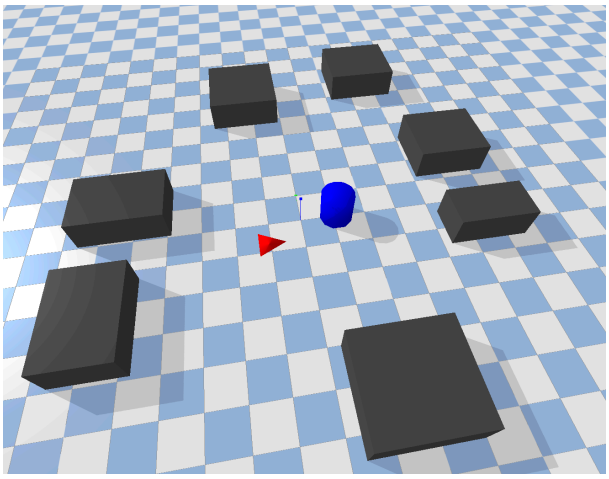
Figure 10: Path comparison (seed 4331): static A* (grey, 22.71 m) vs. Space-Time A* (time-coloured, 23.80 m). The dynamic path detours south near the patrol line to avoid the moving cylinder.

3.3 Task 3.3: Final Execution

The Space-Time A* timed path $[(x, y, t), \ldots]$ is executed in a **PyBullet GUI** simulation. The robot is teleported to each waypoint at the corresponding timestamp while the dynamic obstacle patrols autonomously via `env.update_simulation()`. The robot reaches the goal at $t = 17.5\,\mathrm{s}$ and holds for $2\,\mathrm{s}$ to confirm arrival.



(a) Environment overview: 7 static rectangular obstacles (grey), triangular robot (red), and cylindrical patrol obstacle (blue) moving horizontally.



(b) Robot avoiding the moving cylinder: the robot detours south (visible path) to maintain safe clearance while the cylinder patrols the horizontal corridor.

Figure 11: PyBullet simulation execution (seed 4331). The robot successfully navigates from start to goal, avoiding all static obstacles and the dynamically moving cylinder.

Environment Description. Figure 11a shows the full PyBullet environment for seed 4331: 7 randomly placed static rectangular obstacles (grey boxes), the triangular POLY robot (red polygon), and the cylindrical dynamic obstacle (blue cylinder) patrolling horizontally between $(-8, 0)$ and $(8, 0)$. The robot's start position is $(-7.14, -8.34)$ and the goal is $(7.87, 8.21)$.

Dynamic Obstacle Avoidance. Figure 11b captures a moment during execution where the robot is actively avoiding the moving cylinder. The Space-Time A* planner computed a path that detours south of the patrol line, timing the robot's crossing to occur when the cylinder is at a safe distance. The robot maintains a minimum clearance of 1.15 m (as verified in Figure 8), ensuring collision-free navigation despite the cylinder's continuous motion.

The execution was verified visually: the robot navigates from start to goal, successfully avoiding all 7 static obstacles and the moving cylinder throughout the entire 17.5 s trajectory. The PyBullet physics simulation confirms that the planned path is not only collision-free in the discrete-time model, but also executable in a continuous physics environment.

4 Conclusion

This report presented a complete robot planning pipeline from geometry through execution:

1. **C-space computation** via Minkowski difference (morphological dilation), reducing the problem to a point robot on an occupancy grid.
2. **Grid search** (A* / Weighted A*) and **sampling-based planning** (RRT-Connect), compared across 5 random seeds (4331, 1409, 2824, 5012, 5506). RRT-Connect uses ∼90× fewer *stored vertices* than A* *expanded cells* on average (69 vs. 6 183), noting these memory measures are not strictly equivalent. RRT typically produces 6–16% longer paths, though seeds 4331 and 2824 showed it can occasionally outperform grid-quantised A* in continuous space.
3. **Path smoothing** via natural cubic splines achieving C^2 continuity, verified analytically through curvature profiles.
4. **Dynamic obstacle avoidance** using Space-Time A* in (x, y, t), safely navigating around a patrol obstacle with minimum clearance of 1.15 m.
5. **Physical execution** in PyBullet, demonstrating the full plan-to-execution pipeline.

Version hash: `f0f5c2c361ff1294317fe3554e3918b2` (unmodified `env_factory.py`)